

Course Project*

Abstract—In this paper, a DENSE JOINT model is proposed, which takes advantage of DenseNet’s dense characteristics to extract features adequately. Different models and loss functions are trained on AGIN datasets. Different batch sizes, WSD loss functions and various loss functions are trained and tested on multiple models. The training results are given and analyzed, and the reasons are pointed out.

Index Terms—DenseNet, JOINT, AGIN, SRCC, PLCC.

I. MODEL STRUCTURE

The model Dense JOINT proposed by us considers the impact of technical and rationality perspectives with two independent branches. Compared with the model provided in the course, I use the combination of DenseNet121 and ResNet50 in the Rational branch to achieve better training results. The specific design of my model is as follows, model graph can be seen in Fig 1.

A. The Technical Prior Branch

1) *Data Preprocessing*: I first locate the human-sensitive perceptual artifacts in the photo and block the other parts of the photo while keeping these parts of the picture and rotate them randomly so that they lose local semantic information while keeping the global semantic information. This part of preprocessing code is in patch_partition of utils, which uses the end-to-end model trained by PAL4VST to locate the sensing artifacts, process other parts of the image and store the processed images.

2) *Technical Model Structure*: I use Swin Transformer as a feature extraction model for technical modules. Swin Transformer is a vision model based on the Transformer architecture with excellent performance and flexible construction. To be specific, I used a pre-trained Swin-T model and kept its feature extraction part, removing the classification of the model. The extracted features are then returned to technical score values using two linear layers.

B. The Rationality Perceiving Branch

1) *Data Preprocessing*: We process the input image and convert it to 224×224 dimensions and normalize the values. In order to keep the main content of the image and filter out the influence of some technical factors, we use segmentation smoothing algorithm to obtain the low-frequency mapping of the image. Then the image information of low-frequency mapping is used to extract the features.

2) *Model Structure*: I used Resnet50 and Dense121 models to extract features from the rational part of the image, which can be seen in Fig2. ResNet50 uses a deep residual connection structure, which can effectively alleviate the gradient disappearance problem, so as to extract powerful deep features. DenseNet121, through its dense connectivity, is able to better utilize the features of all previous layers and extract richer feature representations. By combining the features of the two, the model can reduce its dependence on a single architecture and improve its adaptability to different types of images.

In order to make better use of the features extracted from the two different models, I propose two feature fusion methods, the combination of Concat and Squeeze and Excitation modules and the conventional direct addition method. Concat and Squeeze-and-Excitation first combine multiple feature vectors with Cat to get a larger feature vector, then Squeeze-and-Excitation uses global information to reweight the features and restore the original feature size. ADD directly sums the two in the dimension of the original feature, which can maintain the consistency between the features and make the process of gradient transfer more stable. Both showed better performance and scores compared with the models provided in the original course.

This method of multi-model feature fusion helps to improve the generalization ability of the model on different data distributions and enhance the robustness of the model.

3) *JOINT WSD Loss Function*: Combined with WSD loss function, a multilevel network structure is designed to calculate the similarity of image features, which is in Fig3. I performed multistage feature extraction on ResNet50 and DenseNet121 respectively to calculate the difference between the images. I divide ResNet50 and DenseNet121 models into 5 stages, and use L2Pooling for feature map down sampling to enhance the multi-scale representation of features. The feature extraction parts of the two models are independent of each other, but the final feature similarity is synthesized from the calculation results of the two features. Using two different deep learning models reduces the limitations of a single model and enhances the model’s adaptability to different images. The input to the function is the cropped image and the image after low-frequency mapping. Due to the complexity of the calculation model, the calculated loss value will be relatively large, but during the training process, the loss value is still decreasing, which does not affect the training result and reflects the better performance.

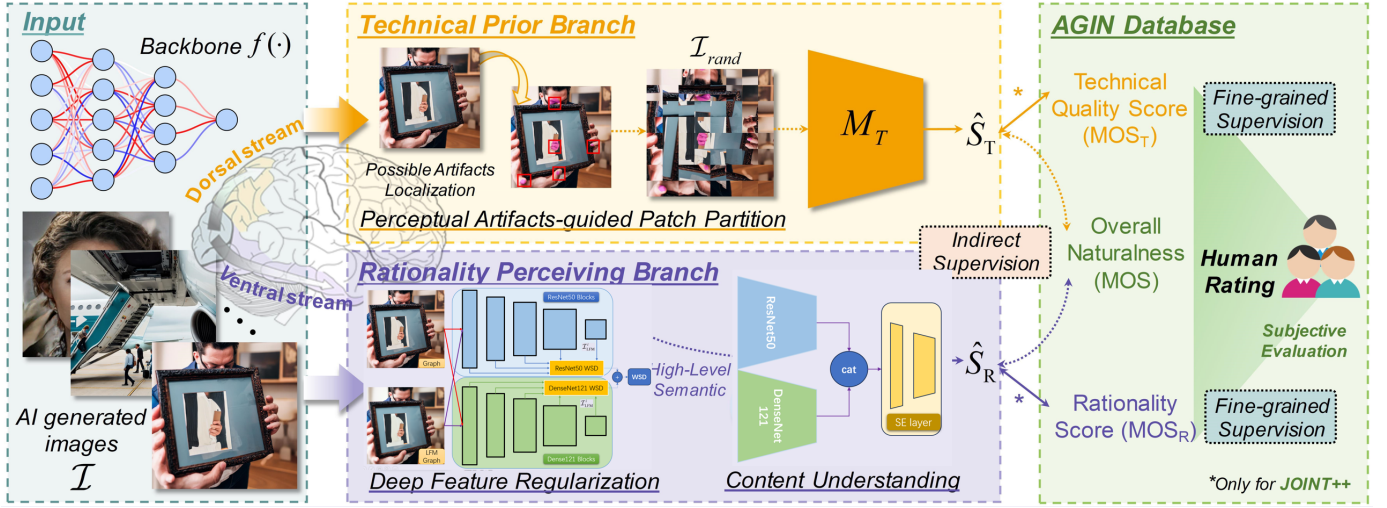


Fig. 1. Dense JOINT Model Structure

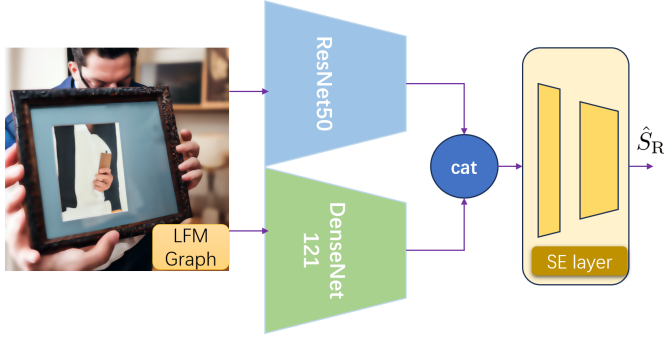


Fig. 2. Rationality Model

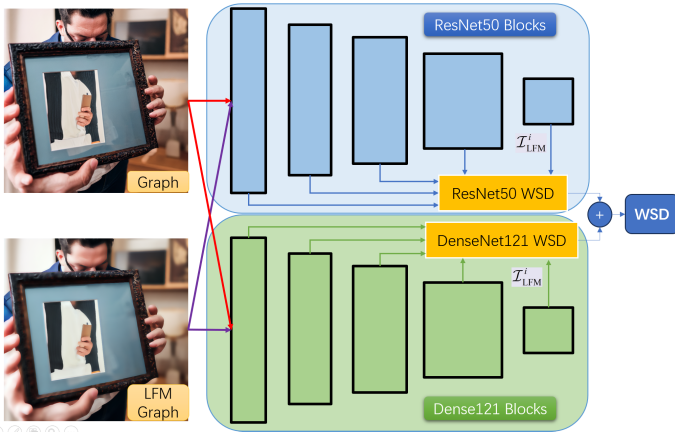


Fig. 3. JOINT WSD Loss

II. EXPERIMENT

In the experiment part, I explored the impact of batch size on model training, the impact of WSD Loss function on model training, the comparison experiment between the model I proposed and the experimental model of the original course, and the ablation experiment using different loss functions.

A. Batch Size Performance

I used different batch sizes to evaluate the scores of the model presented in the original course on different dimensions using the SRCC loss function and the PLCC loss function, and whether the WSD loss function was used in the training with batch sizes of 96 and 32, respectively.

Baseline refers to using the definition of the loss function in the paper and the related hyper parameters also adopt the values in the paper

$$Loss = L_{FS} + \lambda_{IS} L_{IS}$$

$$L_{IS} = L_C(S_T, \hat{MOS}) + L_C(\hat{S}_R, MOS) + \beta \times L_{WSD}$$

$$L_{FS} = L_C(\hat{S}_T, MOS_T) + L_C(\hat{S}_R, MOS_R)$$

$$L_C = L_{MSE} + \alpha \times L_{SRCC}$$

Origin refers to using the definition of the loss function in the original code.

$$Loss = L_{FS}$$

In the baseline loss function mentioned in Table 1 below, the value of L_{WSD} in L_{IS} is 1, while the WSD loss function is used to calculate loss in baseline-WSD. However, L_{IS} is not used in the calculation of origin loss function, so the loss function of origin-WSD is calculated as follows:

$$Loss = L_{FS} + \beta \times L_{WSD}$$

The score of the model test set after training is shown in Table 1. I found that the model with Batch size 32 scored higher on all dimensions than the model with Batch size

TABLE I
Batch Size Performance

Model-Batch Size	Technical		Rationality		Naturalness	
	SRCC $\uparrow$	PLCC $\uparrow$	SRCC $\uparrow$	PLCC $\uparrow$	SRCC $\uparrow$	PLCC $\uparrow$
origin-96	0.7146	0.7615	0.6213	0.6383	0.6854	0.7088
origin-32	0.7242	0.7756	0.6389	0.6601	0.6977	0.7220
baseline-96	0.7187	0.7616	0.6298	0.6475	0.7076	0.7273
baseline-32	0.7318	0.7774	0.6600	0.6763	0.7230	0.7416
baseline_WSD-96	0.7209	0.7639	0.6353	0.6541	0.7086	0.7285
baseline_WSD-32	0.7334	0.7754	0.6516	0.6686	0.7188	0.7380

96. This may be because the smaller batch size allows the model to update the weights more frequently, resulting in more opportunities to learn and update the weights on small data sets. The baseline loss function used in this article is significantly more effective than the origin loss function in the source code, probably because the baseline loss function introduces the concept of indirect supervision compared to Origin, calculating the loss between the target value of each branch and the total output value.

In Fig 4, we can see the reduction in baseline and origin losses under both batch sizes. It can be found that when the batch size is small, the loss decreases slowly, but the final loss will be smaller, so the model performance will be more stable during the training process, and the training effect is better. I didn't use WSD loss function to plot a loss image here because the calculated value of the WSD loss function is proportional to the batch size, so it has no reference value.

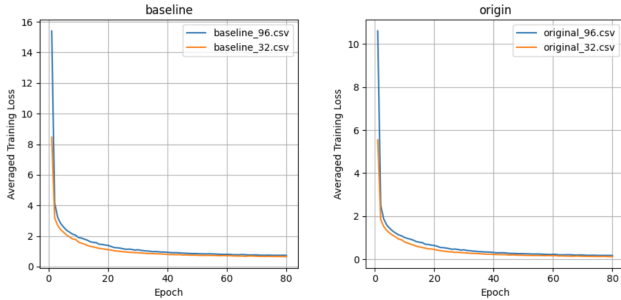


Fig. 4. Batch Size Training Loss

B. Effect of WSD loss function on model

In this section, I investigate the effect of whether the baseline model in the reference paper uses WSD loss functions on the use of SRCC loss functions and PLCC loss functions.

The scores of these models in each dimension are shown in Table 2. I have found that the use of WSD has little effect on Technical scores and in most cases improves the Rationality score because the WSD loss function is calculated using the output of different layers of the model in the rational branch, Therefore, the WSD loss function improves the model's score on Rationality.

Since the size of the loss function is different with and without WSD, I will use the experiment of WSD (same batch size) to plot the same plot, whose training average loss function is shown in Fig 5. Among them, the Loss image of the two images with WSD is very unstable, but the overall trend is downward. In addition, since the Loss calculated by the two loss functions is too small for the loss calculated by WSD, they are not of the same order of magnitude, so the two images using WSD almost coincide visually, but in fact the values of the two are significantly different. In the average training Loss image without WSD Loss function, we can obviously see that loss is gradually declining, and the decline speed of SRCC is faster than that of PLCC, and the final loss is lower, which also corresponds to the better score of SRCC than PLCC in the experimental results.

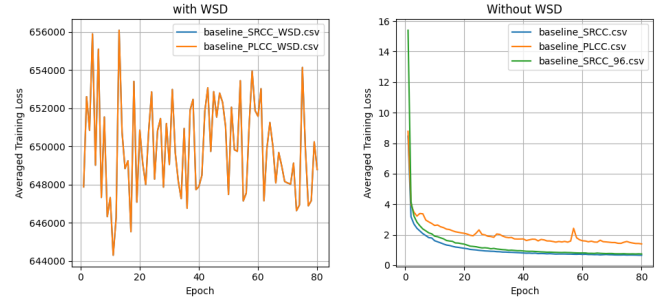


Fig. 5. Effect of WSD loss function on model

C. Model comparison

In this part, I will compare the training effect of the model Dense JOINT(with different feature fusion) I proposed and the model in the original course code with or without WSD loss function on SRCC Loss function.

I proposed two models cat and add, which use different feature fusion of output features of ResNet50 and DenseNet121 modules on the Rationality perceiving branch. The cat model uses concat to splice the features of ResNet50 and DenseNet121, then get the final score with Squeeze-and-Excitation layer. The add model just add the features.

The training results of the model proposed by me and the baseline model are shown in Table 3, from which I can clearly see that the indicators of the model proposed are basically

TABLE II
Effect of WSD loss function on model

Loss Function(Batch Size)	Technical		Rationality		Naturalness	
	SRCC↑	PLCC↑	SRCC↑	PLCC↑	SRCC↑	PLCC↑
SRCC	0.7318	0.7774	0.6600	0.6763	0.7230	0.7416
SRCC WSD	0.7334	0.7754	0.6516	0.6686	0.7188	0.7380
SRCC(96)	0.7187	0.7616	0.6298	0.6475	0.7076	0.7273
SRCC WSD(96)	0.7209	0.7639	0.6353	0.6541	0.7086	0.7285
PLCC	0.7152	0.7613	0.6125	0.6317	0.6990	0.7209
PLCC WSD	0.7027	0.7514	0.6376	0.6517	0.7027	0.7235

higher than the baseline model regardless of whether WSD is used. In addition, both SRCC and PLCC indicators of naturalness improved by more than 1%, and models integrating the two features using WSD and adding fusion methods scored no lower on other dimensions than almost all other models. In particular, the score of rationality dimension has increased by 4% compared with the baseline. Therefore, the proposed model can significantly improve the score of rationality and the score of naturalness without significantly reducing the technical score.

It can be seen in Fig 6 that the average Loss curve of the two models proposed by us dropped the fastest and finally reached a lower value in the training. The characteristics of the average Loss curve also indicate that the model proposed performs better in the training process.

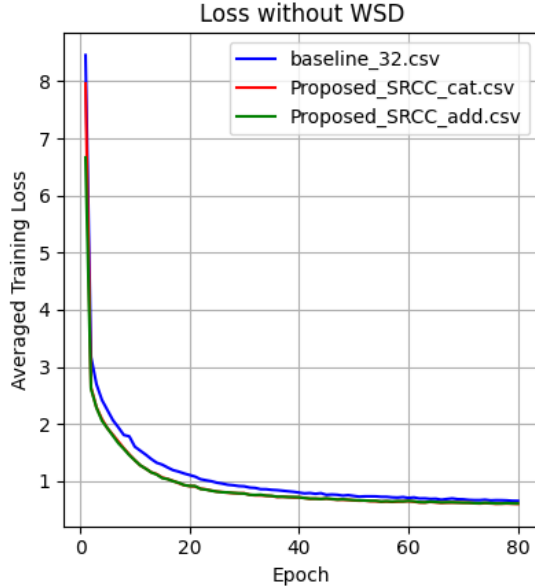


Fig. 6. Effect of WSD loss function on model

D. Comparative experiment of different Loss functions

In this section, I used several different Loss functions to train my proposed model. Let me first reiterate the structure of the loss function that I am currently using.

$$L_{JOINT} = L_{FS} + 0.5 \times L_{IS}$$

In which, L_{IS} uses the overall naturalness MOS via indirect supervision, which is defined as follows:

$$L_{IS} = L_C(S_T, \hat{MOS}) + L_C(\hat{S}_R, MOS) + 0.5 \times L_{WSD}$$

$$L_C = L_{MSE} + 0.5 \times L_{SRCC}$$

L_{FS} is a fine-grained version using the corresponding perspective opinions for both branches:

$$L_{FS} = L_C(\hat{S}_T, MOS_T) + L_C(\hat{S}_R, MOS_R)$$

I replace L_{SRCC} with other loss functions, such as PLCC, L1 loss functions, etc., and compare experiments on different feature fusion modes on baseline and my proposed model. The influence of WSD loss function on model training is tested. The loss functions I test are as follows.

1) Loss functions:

• SRCC loss function

$$L_{SRCC} = 1 - \frac{\sum_n (V_n - \bar{V})(P_n - \bar{P})}{\sqrt{\sum_n (V_n - \bar{V})^2 \sum_n (P_n - \bar{P})^2}}$$

SRCC is defined in the form of Pearson linear correlation coefficient (PLCC) between grades, emphasizing the monotonicity of predictions. Where V and P are the vectors obtained after sorting the original inputs P and V .

• PLCC loss function

$$L_{PLCC} = 1 - \frac{\sum_n (v_n - \bar{v})(p_n - \bar{p})}{\sqrt{\sum_n (v_n - \bar{v})^2 \sum_n (p_n - \bar{p})^2}}$$

PLCC is also defined in the form of Pearson linear correlation coefficient, whose purpose is to maximize the linear correlation between the predicted and true values, such that the higher their correlation, the smaller the loss. Where v and p represent two vectors of input, $\bar{v}$ and $\bar{p}$ represent the mean value of v and p .

TABLE III
Model comparison

Model-Loss function	Technical		Rationality		Naturalness	
	SRCC↑	PLCC↑	SRCC↑	PLCC↑	SRCC↑	PLCC↑
baseline	0.7318	0.7774	0.6600	0.6763	0.7230	0.7416
proposed_add	0.7298	0.7763	0.7074	0.7142	0.7384	0.7528
proposed_cat	0.7242	0.7696	0.7110	0.7186	0.7395	0.7553
baseline-WSD	0.7334	0.7754	0.6516	0.6686	0.7188	0.7380
proposed_add-WSD	0.7307	0.7740	0.7077	0.7176	0.7412	0.7549
proposed_cat-WSD	0.7307	0.7740	0.7030	0.7143	0.7385	0.7541

- **L_1 loss function**

$$L_1 = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

L_1 loss function measures the difference between the predicted value and the true value. L_1 loss is insensitive to outliers. Because it uses absolute values rather than squares, it is less susceptible to extreme errors. Where y_i indicates the true value and $\hat{y}_i$ indicates the predicted value.

- **Cross entropy loss function**

$$L_{Cross} = - \sum_{i=1}^N (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i))$$

The cross entropy loss function measures the difference between the true distribution and the predicted distribution. It directly measures the distance between the probability distribution of the predicted label and the real label, and gives a larger penalty for the larger deviation to help the model predict more accurately. Where y_i indicates the true value and $\hat{y}_i$ indicates the predicted value.

- **Exponential loss function**

$$L_{exp} = \frac{1}{N} \sum_{i=1}^N e^{-y_i \hat{y}_i}$$

Exponential loss function assigns higher weights to large errors, making the model pay more attention to those samples with large errors during training, thus improving the overall prediction performance.

In Table 4, I have marked the loss function used by the model with the highest index in the current model in red, and the number marked in blue is the model and loss function with the highest score for each index in all the different models and loss functions.

2) Table Result Analysis:

- **Technical scores**

For the technical scores, we can see that there is not much difference in the scores between different models using the same loss function, because the model I proposed is the same as the original model in the technical branch,

both of which are feature extractors of the Swin Transformer architecture.

Compared with different loss functions, except PLCC loss function has low scores in the two indexes of the technical branch, other loss functions show approximate scores, and L_1 loss function and cross entropy loss function have better scores in the two indexes than other loss functions. This may be because L_1 loss function is more inclusive to some values with large errors and more robust to outliers during training. The cross entropy loss function will normalize the input value through the activation layer, and use the logarithmic function to measure the gap between the predicted value and the true value, so that the larger error has a larger gradient, so as to effectively guide the model to update the weight. Therefore, L_1 loss function and cross-entropy loss function will help the model to be more inclusive and sensitive to outliers, so that the technical branch module can learn the weight better and have better performance.

Compared with SRCC loss function, PLCC has less monotonic sorting of input content in the implementation, and the input of technical branches is scrambled pictures, and the input content is not standardized, resulting in unstable value of PLCC training and unsatisfactory training effect.

- **Rationality scores**

As for the rationality score, it can be seen that the SRCC loss function gets the highest score in the baseline model, while the model I proposed is more sensitive to the use of the loss function. The model using the add method for feature fusion gets the highest score in the two indexes of rationality when using the L_1 loss function. However, the scores of different loss functions in the two dimensions of the module using cat mode for feature fusion are relatively scattered, but they are obviously better than the baseline model and the gap between them and the model with add mode is very small. Among them, we can see that the model using add fusion mode has an improvement of about 8% compared with the baseline model, and the effect is very obvious. This is mainly because compared with the baseline model, the structure of the rational branch of the model proposed by me has been greatly modified. I used ResNet50 and

TABLE IV
Loss Function Comparasion

Loss function-Model	Technical		Rationality		Naturalness	
	SRCC↑	PLCC↑	SRCC↑	PLCC↑	SRCC↑	PLCC↑
SRCC-Baseline	0.7318	0.7774	0.6600	0.6763	0.7230	0.7416
PLCC-Baseline	0.7152	0.7613	0.6125	0.6317	0.6990	0.7209
L1-Baseline	0.7365	0.7801	0.6560	0.6726	0.7159	0.7359
Cross-Baseline	0.7256	0.7727	0.6550	0.6694	0.7169	0.7366
Exp-Baseline	0.7350	0.7755	0.6484	0.6654	0.7169	0.7350
SRCC WSD-Baseline	0.7334	0.7754	0.6516	0.6686	0.7188	0.7380
PLCC WSD-Baseline	0.7027	0.7514	0.6376	0.6517	0.7027	0.7235
L1-WSD-Baseline	0.7359	0.7782	0.6402	0.6633	0.7100	0.7323
Cross-WSD-Baseline	0.7259	0.7698	0.6471	0.6677	0.7150	0.7351
Exp-WSD-Baseline	0.7256	0.7720	0.6433	0.6598	0.7100	0.7295
SRCC-Proposed ADD	0.7298	0.7763	0.7074	0.7142	0.7384	0.7528
PLCC-Proposed ADD	0.7009	0.7473	0.6715	0.6837	0.7196	0.7380
L1-Proposed ADD	0.7298	0.7753	0.7143	0.7195	0.7463	0.7576
Cross-Proposed ADD	0.7322	0.7732	0.7091	0.7165	0.7448	0.7580
Exp-Proposed ADD	0.7309	0.7713	0.7092	0.7191	0.7480	0.7624
SRCC-Proposed CAT	0.7242	0.7696	0.7110	0.7186	0.7395	0.7553
PLCC-Proposed CAT	0.6962	0.7425	0.6778	0.6919	0.7230	0.7369
L1-Proposed CAT	0.7335	0.7774	0.7107	0.7197	0.7430	0.7570
Cross-Proposed CAT	0.7283	0.7695	0.6914	0.7027	0.7410	0.7562
Exp-Proposed CAT	0.7276	0.7715	0.6931	0.7071	0.7377	0.7552
SRCC WSD-Proposed ADD	0.7307	0.7740	0.7077	0.7176	0.7412	0.7549
PLCC WSD-Proposed ADD	0.6963	0.7531	0.6894	0.6991	0.7276	0.7427
L1 WSD-Proposed ADD	0.7306	0.7743	0.7141	0.7210	0.7436	0.7581
Cross WSD-Proposed ADD	0.7345	0.7786	0.7076	0.7147	0.7455	0.7599
Exp WSD-Proposed ADD	0.7317	0.7736	0.7067	0.7136	0.7422	0.7570
SRCC WSD-Proposed CAT	0.7307	0.7740	0.7030	0.7143	0.7385	0.7541
PLCC WSD-Proposed CAT	0.7242	0.7696	0.7110	0.7186	0.7395	0.7553
L1 WSD-Proposed CAT	0.7269	0.7743	0.6965	0.7059	0.7311	0.7491
Cross WSD-Proposed CAT	0.7345	0.7786	0.6953	0.7043	0.7392	0.7543
Exp WSD-Proposed CAT	0.7310	0.7704	0.6995	0.7067	0.7397	0.7537

DenseNet121 to extract the features of the rational branch and modify the WSD loss function accordingly, so that the model is more sensitive to the rational input and the extracted information is more comprehensive. This effectively improves the score of the rational branch and reduces the gap between the rational score and the technical branch.

In the baseline model, we can see that the score using the SRCC loss function is better than other loss functions. This may be because the features extracted by one model are on the same scale, SRCC sorts them and calculates the loss more fully than the other loss functions, thus resulting in better results.

However, the SRCC loss function is not well trained on the model proposed by me, probably because the features extracted by ResNet50 and DenseNet121 may not be on the same scale, so the effect of directly using the SRCC loss function may not be good. The model I proposed has little difference in effect on L1 loss function, cross entropy loss function and exponential loss function and shows the best performance on L1 loss function. This may be because the features extracted by

the two models are at different scales. The use of L1 loss function is more inclusive of input errors, and the use of cross entropy loss function and exponential loss function will have a higher gradient for those with larger errors, thus better optimizing the parameters of the model. Therefore, Compared with SRCC loss function and PLCC loss function, L1, cross entropy, and exponential loss function can make full use of the fusion characteristics and have a higher score in the model I proposed.

• Naturalness scores

For the naturalness score, we can see that for the baseline model, as in the case of the rationality score, a higher score is achieved using the SRCC loss function. For the model I proposed, the score of using each loss function is higher than that of using the baseline model. Among them, the add fusion method using the exponential loss function has the highest naturalness score, with an increase of about 4% compared with the baseline model. Since the output of the naturalness score model is obtained by adding the technical output and the rational output, the improvement of the model proposed by me is mainly reflected by the improvement in the rational

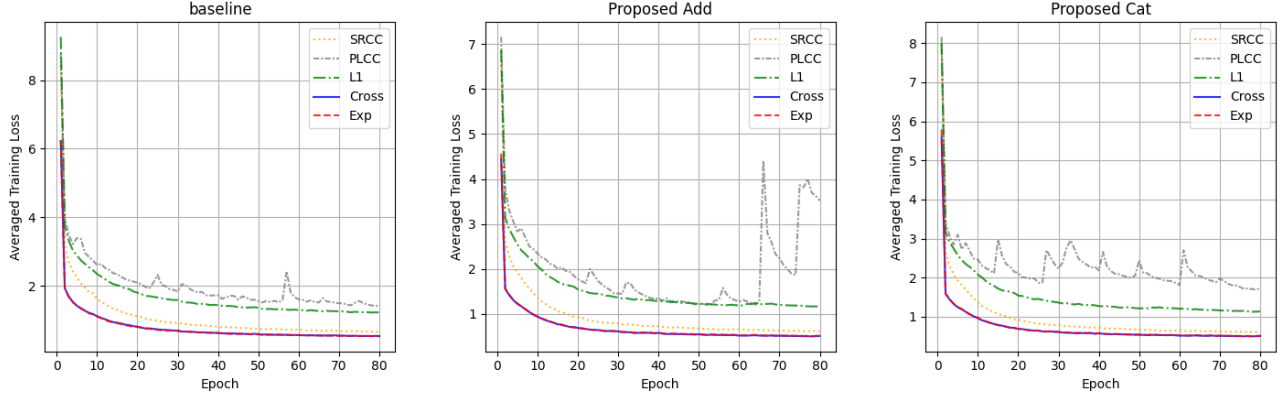


Fig. 7. Effect of WSD loss function on model

branch compared with the baseline model, which mainly adds the DenseNet121 model to extract sufficient model information.

In the baseline model, it can be seen that the use of SRCC loss function has the highest naturalness score. This is because the difference between the use of SRCC loss function and L1 loss function in the technical dimension is not large, and the rational score is better than that of other loss functions. Therefore, the use of SRCC loss function in the baseline model will obtain the highest score. And there is little difference between the naturalness score using L1 loss, which also accords with the characteristic that the output of the naturalness model is added by the output of the previous two dimensions.

In the model I proposed, it can be seen that the model of add feature fusion using exponential loss function shows the best performance. Compared with PLCC loss function and SRCC loss function, the improvement is more obvious, and compared with other loss functions, the improvement is relatively small. However, the effect of cat feature fusion model using exponential function is not much better than the effect of SRCC loss function. This may be because the use of cat in the rational module will increase the dimension of features. Although the SE module will further adjust the weights, more calculation errors will be introduced compared with the fusion method of add, and the exponential loss function is very sensitive to errors, so it will have a greater impact on the fusion method of cat. As a result, it is less effective than using the SRCC loss function. Compared with SRCC and PLCC loss functions, L1 loss functions and Cross loss functions are more tolerant and sensitive to errors, so they have better training results.

3) *Loss Figure Analysis:* In Fig 7, we can see the loss curves of different loss functions in the training process of models with the baseline and the proposed add fusion mode and cat fusion mode. From this, we can see that the loss curves using cross entropy loss function and exponential loss function

are very close, and the amplitude of decline is close to the final loss value, and the decline process is the fastest among all loss functions. This may be because these two loss functions are equally sensitive to input errors and the input with larger errors will have a larger gradient, so that the model parameters are closer in the updating process, resulting in more consistent loss curves for training. When using SRCC loss function, the trend of decline curve is consistent with that of cross entropy and exponential loss function, but the decline speed is not as fast as them. This is because compared with the two loss functions, SRCC calculates ranking correlation, and the smaller gradient amplitude leads to slower model training. Although the loss of L1 loss function decreases more slowly than that of SRCC, its score is better than that of SRCC loss function. This may be because L1 loss function uses absolute value for error calculation, so it is more inclusive to large errors. Therefore, although the loss does not decrease much, the score is better. The training curve of PLCC loss function is relatively stable in the baseline model, but the training curve changes greatly in the model I proposed. Although the implementation of SRCC and PLCC loss function is similar, the stability of SRCC is better than that of PLCC. This may be because PLCC does not sort the input data monotonically during the implementation process, so the calculation error is relatively large. Moreover, PLCC has division in the calculation, and the uncertainty of the calculation is relatively large compared with other loss functions, so the trained loss curve is unstable.

III. CODE IMPLEMENT

A. train

In the train function, I added the functions of continuing training and writing training logs for the loaded model on the basis of the original. The code of this part is in train.py (train_dense.py).

B. test

In the test function, I read the parameters of the saved model and defined the test data set to test the model and returned the

scores of the three dimensions of technicality, Rationality and naturalness respectively. The code of this part is in test.py (test_dense.py).

C. Dense JOINT

Dense JOINT model is based on the JOINT model in the code provided by the original course. The model structure consists of two feature extractors. A Swin Transformer for technical features, Another used rationality features are ResNet-50 and Densenet-121, which I add. In the forward layer, features extracted by ResNet50 and DenseNet121 are fused in two ways (ConCat SElayer and Add). Then the final score is obtained by using the linear regression function composed of two linear layers. The code of this part is in models_densenet/JOINT.py.

D. Densenet121

The model defines a Densenet-121-based deep learning model for the mass regression task. It uses a pre-trained DenseNet-121 model to extract the features of the input image. After the input images in DenseNet pass through an initial convolution layer and the maximum pooling layer, they pass through four dense blocks, and each dense block is followed by an average pooling pool to reduce the number and size of feature maps extracted from dense blocks. Finally, feature vectors extracted by DenseNet are obtained through the global average pooling pool. The feature map is then reduced to a size of 1x1 by means of an adaptive averaging pooling layer. Finally, through the regression module, the features are mapped to the output quality scores. The code for this part is in models_densenet/JOINT.py.

E. SElayer

This module is mainly used to implement the attention mechanism, which adjusts the weight of input features through the learned attention weight, so as to enhance the expression of important features. The structure of the model consists of a global averaging pooling layer to extract the global features of each channel, then generate attention weights through a series of fully connected layers, Swish activation functions, and Sigmoid activation functions, and finally map the adjusted features to the 1000-dimensional output through a linear layer. The code of this part is in models_densenet/se_module.py.

F. PLCC Loss function

This module is mainly used to implement the PLCC (Pearson Linear Correlation Coefficient) loss function by calculating the predicted and true values. I added a very small deviation to the denominator ($1e-8$) to avoid Nan errors caused by division by 0. The code of this part is in utils/function.py.

G. Exponentia Loss function

This module is mainly used to implement the exponentia loss function by calculating the predicted and true values. The code of this part is in utils/function.py.

H. JOINT WSD Loss function

This module is mainly used to implement a joint loss function to enhance the model performance by minimizing the feature difference between the two input images, specifically the weighted loss of the DenseNet and ResNet features. The structure of the model is as follows:

DenseNet121 Initialization: The DenseNet121 model is initialized into five blocks.

- Includes initial convolution layer, batch normalization layer, ReLU activation layer, and maximum pooling layer.
- The middle three sections are dense blocks plus a Transition layer.
- The final section is only one dense block.

ResNet50 Initialization:

- Includes initial convolution layer, batch normalization layer, ReLU activation layer, and maximum pooling layer.
- The Four sections include four ResNet layers of ResNet50.

L2 Pooling Layer:

- L2 pooling for each convolutional layer output.

Feature Extraction:

- **forward_once** method extracts the features of each Layer by DenseNet and ResNet, and adopts a L2 Pooling Layer before each layer is processed.
- **forward** method extracts the features of the two inputs, calculates their feature differences, and calculates the weighted loss by window weighting.

The code of this part is in utils/function.py.

IV. SUMMARY

In this paper, on the basis of the course paper, I propose to use technical branch and rational branch to measure the technical score and rational score of pictures. The technical branch takes the Swin Transfmr model as the basis, while the rational branch takes the RseNet50 and DenseNetNet121 models as the basis for feature extraction, and tries different fusion methods on the extracted features to get a better result. The WSD loss function in the course paper is modified for the modified rational branch, so that it can extract the score of each block in RseNet50 and DenseNetNet121 after the original graph and the low-frequency map, calculate the loss respectively, and then sum. I explored the impact of different batch sizes on model training and analyzed the performance of the original course paper model baseline and the model I proposed under the use of SRCC loss function, PLCC loss function, L1 loss function, cross entropy loss function and exponential loss function. It can be seen from the results that the model I proposed has improved the performance of the model when using various loss functions, and has made great progress in the scores of the three dimensions of technicality, rationality and naturalness.